

De estruturas em C a objetos em TypeScript

Equivalentes de linguagem, conceitos fundamentais e uma introdução prática à Programação Orientada a Objetos (POO).

Foco TypeScript, objetos e POO	Exemplo-base Personagem, vida e ataque
-----------------------------------	---

1. Visão geral: C, JavaScript e TypeScript

C é uma linguagem que trabalha mais perto da memória do computador: estruturas, vetores de caracteres e ponteiros são conceitos centrais. TypeScript é uma camada de tipos sobre JavaScript. Ele ajuda a detectar erros enquanto o programa é desenvolvido, mas o código é transformado em JavaScript para ser executado.

- C enfatiza tipos de memória, ponteiros e controle detalhado.
- JavaScript trabalha naturalmente com objetos, funções e execução em navegador ou Node.js.
- TypeScript acrescenta tipos como string, number e interfaces, tornando projetos maiores mais seguros e fáceis de manter.

2. O exemplo de personagem em TypeScript

```
interface Personagem {  
  nome: string;  
  pontosVida: number;  
}  
  
function ataque(alvo: Personagem, dano: number): void {  
  alvo.pontosVida -= dano;  
  
  if (alvo.pontosVida < 0) {  
    alvo.pontosVida = 0;  
  }  
  
  console.log(`${alvo.nome} recebeu ${dano} de dano! PV restante: ${alvo.pontosVida}`);  
}  
  
const heroi: Personagem = {  
  nome: "Aragorn",  
  pontosVida: 100,  
};  
  
console.log(`Personagem criado: ${heroi.nome} com ${heroi.pontosVida} PV.\n`);  
ataque(heroi, 30);  
ataque(heroi, 80);
```

O programa cria um objeto chamado heroi, exibe seu estado inicial e chama a função ataque duas vezes. A função altera os pontos de vida do mesmo objeto e impede que o resultado fique negativo.

3. Estrutura (struct) em C -> interface em TypeScript

```
// C
typedef struct {
  char nome[50];
  int pontos_vida;
} Personagem;

// TypeScript
interface Personagem {
  nome: string;
  pontosVida: number;
}
```

Em C, struct agrupa campos relacionados em uma única estrutura. Em TypeScript, interface define o formato esperado de um objeto. Ela não cria um objeto por si só; funciona como um contrato que o TypeScript verifica.

Quando utilizar interface: para representar dados que possuem uma forma conhecida, como aluno, produto, pedido, usuário, personagem ou uma resposta recebida de uma API.

```
interface Produto {
  nome: string;
  preco: number;
  emEstoque: boolean;
}

const teclado: Produto = {
  nome: "Teclado",
  preco: 120,
  emEstoque: true,
};
```

4. Tipos básicos: char[] e int -> string e number

4.1 Texto: char nome[50] -> nome: string

Em C, um texto é normalmente um vetor de caracteres. Por isso, char nome[50] reserva espaço para caracteres. Em TypeScript, string representa diretamente um texto; não é necessário copiar caractere a caractere nem definir um tamanho máximo.

```
const nome: string = "Aragorn";
heroi.nome = "Legolas";
```

4.2 Números: int -> number

TypeScript usa number para números inteiros e decimais comuns. Assim, pontos de vida, dano, preço e distância podem ser representados por number.

```
const dano: number = 30;
const distancia: number = 12.5;
```

Aplicação: use string para nomes, mensagens, códigos e endereços; use number para contagens, valores, medidas, notas e cálculos.

5. Criação de dados: variável estruturada -> objeto

```
// C: declarar e preencher em etapas
Personagem heroi;
strcpy(heroi.nome, "Aragorn");
heroi.pontos_vida = 100;

// TypeScript: criar o objeto já preenchido
const heroi: Personagem = {
  nome: "Aragorn",
  pontosVida: 100,
};
```

Um objeto reúne propriedades que descrevem uma entidade. Nesse caso, heroi tem duas propriedades: nome e pontosVida. O trecho : Personagem pede ao TypeScript que confira se todas as propriedades exigidas pela interface existem e têm o tipo correto.

const e let

- Use const quando a variável não deve apontar para outro objeto. As propriedades internas ainda podem ser alteradas.
- Use let quando será necessário atribuir outro valor ou outro objeto à variável.
- Evite var em código TypeScript moderno.

```
const heroi: Personagem = { nome: "Aragorn", pontosVida: 100 };
heroi.pontosVida = 70; // permitido
// heroi = outroHerói; // não permitido com const
```

6. Funções: void, parâmetros e retorno

```
function ataque(alvo: Personagem, dano: number): void {
  // comandos da função
}
```

- function ataque: declara uma função com o nome ataque.
- alvo: Personagem: o primeiro argumento precisa ser um objeto compatível com a interface Personagem.
- dano: number: o segundo argumento precisa ser numérico.
- : void: a função executa uma ação, mas não devolve um resultado com return.

Use void para funções que alteram algo, imprimem informações, salvam dados ou notificam o usuário. Use um tipo de retorno, como number, string ou boolean, quando a função precisa calcular e devolver uma informação.

```
function calcularDanoFinal(danoBase: number, defesa: number): number {
  return Math.max(0, danoBase - defesa);
}
```

7. Ponteiro e acesso a propriedades

```
// C
alvo->pontos_vida -= dano;

// TypeScript
alvo.pontosVida -= dano;
```

Em C, `Personagem *alvo` é um ponteiro; o operador `->` acessa um campo por meio desse endereço. TypeScript não usa essa sintaxe. Quando um objeto é passado a uma função, a função recebe acesso ao mesmo objeto, portanto consegue alterar suas propriedades.

Consequência prática: ao chamar `ataque(heroi, 30)`, a modificação feita em `alvo.pontosVida` aparece em `heroi.pontosVida` depois que a função termina.

Atenção: valores simples, como `number`, `string` e `boolean`, não se comportam da mesma forma quando são passados a uma função. Alterar apenas o parâmetro não altera a variável externa.

8. Operadores e decisões

8.1 -=

```
alvo.pontosVida -= dano;  
// equivale a:  
alvo.pontosVida = alvo.pontosVida - dano;
```

8.2 if

```
if (alvo.pontosVida < 0) {  
  alvo.pontosVida = 0;  
}
```

`if` executa um bloco de código somente quando a condição é verdadeira. Aqui, ele protege uma regra do domínio do jogo: vida não pode ser negativa. Esse tipo de validação é comum em estoque, saldo, vagas disponíveis, notas e limites de crédito.

8.3 console.log e template strings

```
console.log(`${alvo.nome} recebeu ${dano} de dano!`);
```

`console.log` mostra uma informação no terminal. As crases formam uma template string; `${...}` insere valores dentro do texto. É mais legível do que montar mensagens com muitas concatenações.

9. Relação com POO

A interface e o objeto do exemplo organizam dados, mas ainda não formam uma classe completa. Em POO, normalmente agrupamos os dados e os comportamentos que pertencem a uma entidade. Para um personagem, receber dano é um comportamento natural do próprio personagem.

9.1 Classe

Uma classe é um molde para criar objetos. Ela pode definir atributos, métodos e regras de funcionamento. O objeto criado a partir de uma classe é chamado de instância.

```
class Personagem {  
  nome: string;  
  pontosVida: number;  
  
  constructor(nome: string, pontosVida: number) {  
    this.nome = nome;  
    this.pontosVida = pontosVida;  
  }  
  
  receberAtaque(dano: number): void {  
    this.pontosVida -= dano;  
  }  
}
```

```

if (this.pontosVida < 0) {
  this.pontosVida = 0;
}

console.log(`${this.nome} recebeu ${dano} de dano! PV restante: ${this.pontosVida}`);
}
}

const heroi = new Personagem("Aragorn", 100);
heroi.receberAtaque(30);

```

9.2 this

Dentro de uma classe, `this` representa o próprio objeto que está executando o método. Assim, `this.nome` é o nome do personagem que recebeu o ataque, e `this.pontosVida` é a vida dele.

9.3 constructor

constructor é executado automaticamente quando usamos `new`. Ele é usado para definir o estado inicial da instância e garantir que ela seja criada com dados válidos.

10. Encapsulamento: protegendo a regra de negócio

Em POO, encapsulamento significa controlar como os dados internos podem ser alterados. Em vez de permitir que qualquer parte do programa defina uma vida negativa, podemos tornar o atributo privado e fornecer métodos para realizar mudanças válidas.

```

class Personagem {
  constructor(
    public nome: string,
    private pontosVida: number
  ) {}

  receberAtaque(dano: number): void {
    this.pontosVida = Math.max(0, this.pontosVida - dano);
  }

  obterPontosVida(): number {
    return this.pontosVida;
  }
}

const heroi = new Personagem("Aragorn", 100);
heroi.receberAtaque(30);
console.log(heroi.obterPontosVida()); // 70

```

- `public`: pode ser acessado fora da classe; é o padrão em TypeScript.
- `private`: só pode ser acessado dentro da própria classe.
- Métodos públicos são uma forma de expor ações seguras, como `receberAtaque` ou `obterPontosVida`.

11. Quando usar interface e quando usar class

Recurso	Use quando	Exemplo
interface	Você precisa descrever o formato de dados e validar objetos.	Produto, Usuario, Personagem recebido de uma API.

Recurso	Use quando	Exemplo
class	Você precisa criar objetos com comportamento, métodos e regras.	Personagem que ataca, conta bancária que deposita, carrinho que calcula total.
type	Você precisa combinar tipos ou criar alternativas simples.	type Status = 'ativo' 'inativo';

12. Aplicações práticas

- Jogos: personagens, inimigos, inventários, habilidades e regras de dano.
- Sistemas escolares: Aluno, Disciplina, Nota e métodos para calcular média.
- Lojas virtuais: Produto, Carrinho e Pedido, com regras de estoque e desconto.
- Sistemas bancários: Conta, Cliente e operações como sacar, depositar e transferir.
- APIs e front-end: interfaces para garantir a forma dos dados recebidos do servidor.

13. Passo a passo para executar

- Salve o código em um arquivo chamado personagem.ts.
- No terminal, execute `npx ts-node personagem.ts`, se o `ts-node` estiver configurado.
- Como alternativa, compile com `npx tsc personagem.ts` e execute o JavaScript gerado com `node personagem.js`.
- Observe as mensagens no terminal e teste outros valores de dano.

14. Resumo dos equivalentes

Em C	Em TypeScript
<code>typedef struct ... Personagem</code>	<code>interface Personagem { ... }</code>
<code>char nome[50]</code>	<code>nome: string</code>
<code>int pontos_vida</code>	<code>pontosVida: number</code>
<code>Personagem *alvo</code>	<code>alvo: Personagem</code>
<code>alvo->campo</code>	<code>alvo.campo</code>
<code>void funcao(...)</code>	<code>function funcao(...): void</code>
<code>printf(...)</code>	<code>console.log(...)</code>
<code>strcpy(...)</code>	atribuição direta de texto

Conclusão

A conversão de C para TypeScript troca detalhes de baixo nível, como vetores de caracteres e ponteiros explícitos, por recursos mais diretos: strings, objetos, interfaces e funções tipadas. Ao evoluir o exemplo para uma classe, os dados do personagem e as regras para modificá-los ficam reunidos no mesmo lugar - princípio central da POO.